Installation and Testing of Jasmin and EasyCrypt on the Raspberry Pi 5

A reproducible procedure for the formal verification of cryptographic code on AArch64

October 21, 2026

Abstract

This guide describes, step by step, how to install the *Jasmin* compiler (v2026.09.0) and the *EasyCrypt* proof assistant (r2026.07) on a Raspberry Pi 5 (8 GB RAM, Debian 12 "Bookworm", AArch64), and how to confirm with four tests, comprising 11 checks, that the toolchain works. It is written for readers who are familiar with the Linux command line but have not used Jasmin, EasyCrypt, OCaml or Coq before; it explains each concept when it first appears. Each executed command appears in a listing together with the relevant part of its output. The tests demonstrate four capabilities of the toolchain: (i) the compilation of a Jasmin program to native ARMv8-A assembly and its execution against a C reference implementation; (ii) the verification of constant-time security with `jasmin-ct`; (iii) the checking of EasyCrypt proofs, including a probabilistic relational proof; and (iv) the translation of Jasmin code to EasyCrypt with `jasmin2ec` and the proof of its functional correctness. All 11 checks pass. The guide also documents four problems that arose during the installation (a full SD card, a mislabeled Z3 release file, glibc incompatibilities, and an operand restriction of the ARMv8-A back end) and their resolutions.

Contents

1 Introduction **3**
- 1.1 Purpose and audience . . . 3
- 1.2 The tools . . . 3
- 1.3 Overview of the procedure . . . 3
- 1.4 Conventions . . . 4

2 Test platform **4**

3 Preparation: storage and memory **4**

4 System packages **5**

5 opam and the OCaml switch **6**

6 Installation of Jasmin **6**
- 6.1 Selection of versions . . . 6
- 6.2 Installation with opam . . . 7
- 6.3 Verification of the installation . . . 7

7 Installation of EasyCrypt **8**
- 7.1 Installation with opam . . . 8
- 7.2 Configuration of the SMT solvers . . . 8

8 Tests **9**
8.1 Test 1: compilation of Jasmin to ARMv8-A and native execution 10
8.2 Test 2: constant-time verification with `jasmin-ct` 12
8.3 Test 3: EasyCrypt proofs 13
8.4 Test 4: translation from Jasmin to EasyCrypt and correctness proof 15
8.5 Execution of all tests 16

9 Summary **18**

A Complete command sequence **18**

B Glossary **19**

C References **20**

1 Introduction

1.1 Purpose and audience

Cryptographic software must be correct for every input and must not reveal secrets through side channels such as its execution time. Testing alone cannot establish such properties, because it can examine only a small fraction of the possible inputs. *Formal verification* establishes them with machine-checked mathematical proofs. Jasmin and EasyCrypt are two tools that make this approach practical for cryptographic code. This guide installs both tools on a Raspberry Pi 5 and demonstrates their use on small examples. It is the first of three documents in this repository; the second document tests the ARMv8-A back end of the Jasmin compiler, and the third applies both tools to a post-quantum key encapsulation mechanism.

The guide assumes familiarity with the Linux command line (shell commands, packages, environment variables). It does not assume prior knowledge of the programming language OCaml, of the proof assistant Coq, or of Jasmin and EasyCrypt. A glossary of the technical terms is provided in Appendix B.

1.2 The tools

Jasmin is a programming language and a compiler for cryptographic implementations that must be both fast and trustworthy (*high assurance*). Jasmin programs are written at the level of assembly: they name registers, processor flags and memory accesses explicitly, so that the programmer controls the generated machine code. The Jasmin compiler is *formally verified*: a proof, checked by the proof assistant Coq, establishes that the assembly code that the compiler generates has the same behavior as the source program. The Jasmin toolchain also includes checkers for *memory safety* (the absence of accesses outside the allocated memory) and for *constant-time security*. A program is constant-time if secret data influence neither its control flow (which branches it takes) nor the memory addresses that it accesses; otherwise, an attacker who measures the execution time can learn information about the secrets. *Speculative* constant-time security extends this property to the speculative execution of modern processors (attacks of the Spectre family).

EasyCrypt is an interactive proof assistant for cryptographic proofs: the user states a property, and EasyCrypt checks a proof of it that the user writes step by step. EasyCrypt reasons about programs written in a simple imperative language that may draw random values. It supports *Hoare logic*, which relates the states before and after the execution of a program, and *probabilistic relational Hoare logic* (pRHL), which relates the output distributions of two programs; pRHL is the basis of game-based security proofs, in which a real system is shown to be indistinguishable from an idealized one. EasyCrypt delegates routine arithmetic proof steps to automatic provers, called *SMT solvers* (Alt-Ergo, CVC5, Z3), through the intermediate platform *Why3*.

The two tools are designed to work together. The translator `jasmin2ec` converts a Jasmin program into an EasyCrypt module, on which functional correctness and security properties can then be proved. Because the compiler preserves the semantics of the program, these properties also hold for the generated assembly code.

Jasmin v2026.09.0 introduced a **back end for ARMv8-A (AArch64)**, the 64-bit architecture of the processor of the Raspberry Pi 5 (a Cortex-A76); a back end is the part of a compiler that generates code for one particular architecture. Jasmin code can therefore be compiled, executed, tested and verified on the same machine. This guide demonstrates each of these steps.

1.3 Overview of the procedure

Table 1 lists the steps of the installation. Most of the time is spent compiling software from source; the commands themselves are few, and Appendix A collects them in one listing.

Step	Section	Action	Time
1	3	Free storage and check memory	a few min
2	4	Install system packages with `apt-get`	2 min
3	5	Initialize opam and create an OCaml environment (switch)	12 min
4	6	Install Jasmin with opam	16 min
5	7	Install EasyCrypt and the SMT solvers, and configure them	57 min
6	8	Run the four tests (11 checks)	15 s

Table 1: Steps of the installation and their duration on the Raspberry Pi 5.

1.4 Conventions

In shell listings, a line that begins with `$` is a command that the user types; the remaining lines are the output of the command, which is sometimes abridged. The marker `...` indicates an omission. A comment that begins with `#` explains a command and is not part of it. The example files are located in the directory `examples/` next to this document; file paths in this guide are relative to the directory `install-guide/` unless they begin with `/` or `~`.

2 Test platform

Table 2 lists the hardware and software with which every command of this guide was executed. Other versions may work, but they have not been tested.

Item	Value
Board	Raspberry Pi 5 Model B Rev 1.0, 4 cores (Cortex-A76), 7.9 GiB RAM
Storage	64 GB microSD card (`/dev/mmcblk0p2`, ext4)
Operating system	Debian GNU/Linux 12 (bookworm), Raspberry Pi OS kernel 6.12.93+rpt-rpi-2712, aarch64
C compiler	GCC 12.2.0 (Debian 12.2.0-14+deb12u1)
opam / OCaml	opam 2.1.2 (Debian package), OCaml 4.14.2 (switch `jasmin-ec`)
Jasmin	2026.09.0 (`jasminc`, `jasmin2ec`, `jasmin-ct`, `jasmin2tex`)
EasyCrypt	r2026.07 (git tag, pinned in opam)
Why3	1.8.2
SMT solvers	Alt-Ergo 2.6.0 (opam), CVC5 1.0.3 (apt), Z3 4.8.12 (apt)
LaTeX	pdfTeX 3.141592653-2.6-1.40.24 (TeX Live 2022/Debian)

Table 2: Hardware and software of the test platform.

3 Preparation: storage and memory

A build of EasyCrypt and Jasmin from source requires several gigabytes of storage: the final installation occupies 4.0 GB, and the build process needs additional temporary space. The command `df` reports the free storage, and the command `free` reports the free memory:

```
$ df -h ~ | tail -1
/dev/mmcblk0p2   58G   55G  514M 100% /
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.9Gi       5.2Gi       323Mi        14Mi       2.9Gi       2.7Gi
Swap:          2.5Gi       2.5Gi       288Ki
```

On the test platform, the SD card was full (100 % used). Storage was reclaimed by deleting only caches and other files that their programs regenerate on demand: the caches of npm, apt and pip, and superseded versions of the VS Code Remote server, which VS Code installs on the Raspberry Pi when a remote editing session connects to it. The server version in use and the most recently used one were retained. Readers who do not use these programs can skip the corresponding commands; the general rule is to delete only files that are known to be regenerated.

```
$ ps aux | grep -o 'servers/Stable-[0-9a-f]*' | sort | uniq -c   # which server is in use?
     30 servers/Stable-6d62b1d5dace93a33b3a6d3eca1317bd5e15bcb1
$ cd ~/.vscode-server/cli/servers && rm -rf Stable-<unused hashes...>
$ npm cache clean --force
$ sudo apt-get clean
$ rm -rf ~/.cache/pip
$ df -h / | tail -1
/dev/mmcblk0p2   58G   47G  8.2G  86% /
```

In the second command, `<unused hashes...>` stands for the directory names of the server versions that the first command does not report as in use.

> **Note.** Reserve at least **6–8 GB** of free storage. The available memory (1.6–3 GB, with other programs running) sufficed for opam's default parallelism of 3 build jobs on 4 cores. If the operating system terminates a build because memory is exhausted (a message from the *out-of-memory killer* in the output of `dmesg`), rerun `opam install` with the option `-j 1`, which builds one package at a time.

4 System packages

Jasmin and EasyCrypt are installed with opam (Section 5), but they require several programs and libraries from the operating system. Table 3 explains the role of each Debian package; the following command installs them all:

```
$ sudo apt-get update
$ sudo DEBIAN_FRONTEND=noninteractive apt-get install -y \
    opam build-essential m4 pkg-config unzip bubblewrap \
    libgmp-dev libpcre3-dev zlib1g-dev libmpfr-dev libppl-dev autoconf \
    z3 cvc5
$ opam --version
2.1.2
$ z3 --version
Z3 version 4.8.12 - 64 bit
$ cvc5 --version | head -1
This is cvc5 version 1.0.3
```

> **Caution.** The EasyCrypt documentation recommends Z3 4.12.x. The official release file `z3-4.12.4-arm64-glibc-2.35.zip`, however, contains an *x86-64* program instead of an arm64 program, so that its execution fails:
>
> ```
> x86_64-binfmt-P: Could not open '/lib64/ld-linux-x86-64.so.2': No such file or directory
> ```
>
> All later arm64 releases for Linux (4.15.x–5.3.x) require version 2.38 of the C library glibc, whereas Bookworm provides version 2.36. This installation therefore retains Debian's Z3 4.8.12 and uses Alt-Ergo 2.6.0, the version that EasyCrypt recommends, as the principal prover. All tests in this guide pass with this configuration. The opam package `z3` can be built from source instead, but the build takes a long time on the Raspberry Pi.

Package	Role
`opam`	package manager for OCaml software, through which Jasmin and EasyCrypt are installed
`build-essential`, `m4`, `pkg-config`, `autoconf`, `unzip`	C compiler, linker and build tools needed to compile packages from source
`bubblewrap`	sandboxing tool that opam uses by default (disabled in Section 5)
`libgmp-dev`, `libmpfr-dev`, `libppl-dev`	arbitrary-precision arithmetic and polyhedra libraries, required by Apron, the abstract-interpretation library of Jasmin's safety checker
`libpcre3-dev`, `zlib1g-dev`	regular-expression and compression libraries, installed as a precaution; none of the opam packages of this guide requires them, so they may be omitted
`z3`, `cvc5`	two SMT solvers that EasyCrypt can call

Table 3: Debian packages and their roles.

5 opam and the OCaml switch

Jasmin and EasyCrypt are written in OCaml and are distributed through *opam*, the package manager of the OCaml ecosystem. opam installs packages into a *switch*: an independent directory under `~/.opam` with its own OCaml compiler and its own set of packages. Separate switches allow incompatible versions of software to coexist; the third document of this repository, for example, uses a second switch for a development version of Jasmin.

This installation uses a dedicated switch named `jasmin-ec` with OCaml 4.14.2. The first command initializes opam: the option `--bare` prevents opam from creating a default switch, and the option `--disable-sandboxing` prevents failures of `bubblewrap` in containers and remote sessions. The second command creates the switch and compiles the OCaml compiler from source. The third command activates the switch in the current shell, by setting environment variables such as `PATH`.

```
$ opam init -y --bare --disable-sandboxing
...
[WARNING] Shell not updated in non-interactive mode: use --shell-setup
$ opam switch create jasmin-ec ocaml-base-compiler.4.14.2 -y
...
-> installed ocaml-base-compiler.4.14.2
-> installed ocaml-config.2
-> installed ocaml.4.14.2
Done.
$ eval $(opam env --switch=jasmin-ec)
$ ocaml -version
The OCaml toplevel, version 4.14.2
```

The compilation of OCaml took about 12 minutes on the Raspberry Pi 5. The activation applies only to the current shell. To activate the switch in every new shell, add the `eval` line to `~/.zshrc` or `~/.bashrc`, or run `opam init --shell-setup` once. All commands in the remainder of this guide assume an active switch.

6 Installation of Jasmin

6.1 Selection of versions

The Jasmin toolchain translates programs into EasyCrypt code (Section 8.4), and each Jasmin release generates code for one specific EasyCrypt release; the two versions must therefore be

selected together. The following commands retrieve the latest releases from GitHub and the compatibility statement in Jasmin's change log (`CHANGELOG.md`):

```
$ curl -s 'https://api.github.com/repos/jasmin-lang/jasmin/releases?per_page=1' | grep tag_name
    "tag_name": "v2026.09.0",
$ curl -s 'https://api.github.com/repos/EasyCrypt/easycrypt/releases?per_page=1' | grep tag_name
    "tag_name": "r2026.10",
$ curl -s https://raw.githubusercontent.com/jasmin-lang/jasmin/v2026.09.0/CHANGELOG.md \
    | grep -n 'targets version' | head -1
42:- Extraction as EasyCrypt code targets version 2026.07
```

The latest EasyCrypt release (r2026.10) is thus newer than the release that Jasmin 2026.09.0 supports (r2026.07). Table 4 records the selection.

Tool	Latest release	Installed	Reason
Jasmin	v2026.09.0	2026.09.0	first release with the ARMv8-A target
EasyCrypt	r2026.10	r2026.07	version targeted by Jasmin 2026.09.0

Table 4: Selected versions.

6.2 Installation with opam

The first command lists the versions of Jasmin that opam offers; the second installs version 2026.09.0 and measures the duration with `time`:

```
$ opam show jasmin --field=all-versions
2022.04.0  2022.09.0 ... 2026.03.2  2026.09.0
$ time opam install -y jasmin.2026.09.0
...
-> installed apron.v0.9.15
-> installed jasmin.2026.09.0
Done.
<><> conf-gmp-paths.1 installed successfully <><><><><><><><><><><><><><><><><><><><><><>
=> header file found in /usr/include/aarch64-linux-gnu
=> library file found in /usr/lib/aarch64-linux-gnu
opam install -y jasmin.2026.09.0  755.07s user 96.10s system 87% cpu 16:14.07 total
```

The Coq proof of the compiler is not checked during this installation: the opam package contains the OCaml code that was extracted from the Coq development, so the installation does not require Coq. The build took 16 minutes.

6.3 Verification of the installation

The package installs four programs, which Table 5 describes. The following commands confirm that they are available and that the compiler supports the ARMv8-A target (`armv8a`):

```
$ which jasminc jasmin2ec jasmin-ct jasmin2tex
/home/code-j/.opam/jasmin-ec/bin/jasminc
/home/code-j/.opam/jasmin-ec/bin/jasmin2ec
/home/code-j/.opam/jasmin-ec/bin/jasmin-ct
/home/code-j/.opam/jasmin-ec/bin/jasmin2tex
$ jasminc -version
Jasmin Compiler 2026.09.0
$ jasminc -help | grep -A1 -- '-arch'
  -arch {x86-64|arm-m4|armv8a|riscv}
                            Select target arch (default is x86-64)
```

Program	Function
`jasminc`	compiler: translates a Jasmin program (`.jazz`) into assembly for the selected target
`jasmin-ct`	constant-time checker: verifies that secret data influence neither branches nor memory addresses
`jasmin2ec`	translator: converts a Jasmin program into an EasyCrypt module
`jasmin2tex`	pretty-printer: typesets a Jasmin program in LaTeX (not used in this guide)

Table 5: Programs of the Jasmin package.

The package also installs Jasmin's EasyCrypt support library, called *eclib* (`JModel_armv8a.ec`, `JWord.ec`, ...), in the directory `$OPAM_SWITCH_PREFIX/lib/easycrypt/jasmin`. This library defines, in EasyCrypt, the machine words and the instructions that the translated programs use. EasyCrypt locates it automatically (Section 7.2).

7 Installation of EasyCrypt

7.1 Installation with opam

The default opam repository does not provide EasyCrypt at the required version. The command `opam pin` therefore associates the package name `easycrypt` with a specific source, the git tag `r2026.07` of the EasyCrypt repository; the subsequent `opam install` builds the package from this source, together with the SMT solver Alt-Ergo 2.6.0:

```
$ opam pin add -yn easycrypt https://github.com/EasyCrypt/easycrypt.git#r2026.07
[easycrypt.~dev] synchronised (git+https://github.com/EasyCrypt/easycrypt.git#r2026.07)
easycrypt is now pinned to git+https://github.com/EasyCrypt/easycrypt.git#r2026.07 (version ~dev)
$ time opam install -y easycrypt alt-ergo.2.6.0
...
$ opam list --installed easycrypt why3 alt-ergo bitwuzla-cxx
# Name       # Installed # Synopsis
alt-ergo     2.6.0       The Alt-Ergo SMT prover
bitwuzla-cxx 0.9.1       SMT solver for AUFBVFP (C++ API)
easycrypt    ~dev        pinned to version ~dev at git+https://github.com/EasyCrypt/easycrypt.git
    ↪ #r2026.07
why3         1.8.2       Why3 environment for deductive program verification
```

This step took about **57 minutes**. Most of the time was spent on the C++ build of the SMT solver Bitwuzla (`bitwuzla-cxx`), on Why3 and Alt-Ergo, and on a rebuild of Jasmin: the dependencies of EasyCrypt require version 1.3.0 of the OCaml library `cmdliner` instead of version 2.1.1, so opam recompiled every installed package that depends on `cmdliner`, including Jasmin.

> **Note.** The terminal session was disconnected during this step. The `opam install` process continued in the background and completed normally. In such a case, check with `pgrep -af opam` whether an opam process is still running before starting another installation in the same switch: opam locks the switch while it works, and a second `opam install` waits until the lock is released.

7.2 Configuration of the SMT solvers

EasyCrypt calls the SMT solvers through Why3, which must know where each solver is installed. The command `easycrypt why3config` searches for the installed solvers and writes this information to a configuration file; it must be rerun after every installation or update of a solver. The command `easycrypt config` then displays the resulting configuration:

```
$ easycrypt why3config
...
```

```
Found prover Z3 version 4.8.12, OK.
Found prover Z3 version 4.8.12 (alternative: noBV)
10 prover(s) added
Save config to /home/code-j/.config/easycrypt/why3.conf
$ easycrypt config
git-hash: r2026.07
load-path:
  Jasmin@/home/code-j/.opam/jasmin-ec/lib/easycrypt/jasmin
  <system>@/home/code-j/.opam/jasmin-ec/lib/easycrypt/theories
  ...
why3 configuration file
  /home/code-j/.config/easycrypt/why3.conf
...
known provers: Alt-Ergo@2.6.0, Alt-Ergo[BV]@2.6.0, Alt-Ergo[counterexamples]@2.6.0,
  CVC5@1.0.3, CVC5[counterexamples]@1.0.3, CVC5[strings]@1.0.3,
  CVC5[strings+counterexamples]@1.0.3, Z3@4.8.12, Z3[counterexamples]@4.8.12, Z3[noBV]@4.8.12
```

Two entries of this output are relevant. The entry `known provers` lists the solvers that EasyCrypt can invoke; several solvers appear more than once, in different configurations. The entry `Jasmin@. . .` of the load path shows that the Jasmin eclib is registered under the logical name `Jasmin`; the EasyCrypt statement `from Jasmin require import JModel_armv8a.` therefore finds the library without additional options.

8 Tests

The tests confirm that each component of the toolchain works, and they introduce the basic use of each tool. Table 6 summarizes what each test demonstrates. The test files are located in `examples/`, and a single script runs all tests (Section 8.5). The directory has the following structure:

```
examples/
|-- run-tests.sh              # runs all 11 checks
|-- run-tests-output.txt      # output of the last run
|-- jasmin/
|   |-- add256.jazz           # 256-bit addition (ARMv8-A)
|   |-- test_add256.c         # C harness, 10^6 random tests
|   `-- ct_select.jazz        # constant-time and leaky selection
`-- easycrypt/
    |-- hello.ec              # SMT smoke test
    |-- otp.ec                # one-time pad, pRHL proof
    |-- wrong.ec              # negative test: must fail
    |-- CtSelect.ec           # generated by jasmin2ec
    `-- ct_select_proof.ec    # correctness proof of the Jasmin code
```

Test	What the test demonstrates	Tools
1	Jasmin code compiles to ARMv8-A assembly that runs on the Raspberry Pi and computes correct results	`jasminc`, `gcc`
2	The constant-time checker accepts code without secret-dependent branches and rejects code with such a branch	`jasmin-ct`
3	EasyCrypt accepts correct proofs, including a probabilistic one, and rejects an attempted proof of a false statement	`easycrypt`
4	A Jasmin function, translated to EasyCrypt, can be proved functionally correct	`jasmin2ec`, `easycrypt`

Table 6: Overview of the tests. The test script (Section 8.5) divides these four tests into 11 checks.

8.1 Test 1: compilation of Jasmin to ARMv8-A and native execution

The Jasmin program of the first test adds two 256-bit integers. A 64-bit processor cannot add such numbers in one instruction, so each number is represented by four 64-bit words, called *limbs*, and the limbs are added from the least to the most significant one. Each addition sets the processor's *carry flag* when its result exceeds 64 bits, and the next addition adds this carry. Multi-precision addition of this kind is a basic building block of elliptic-curve and other public-key arithmetic. The program manipulates the carry flag explicitly, through the Boolean variable `cf`. The `for` loop assigns the values 1, 2 and 3 to the compile-time variable `i` of type `inline int` (the upper bound 4 is excluded), and the compiler unrolls the loop completely.

```
// 256-bit addition with carry propagation (four 64-bit limbs),
// written for the ARMv8-A (AArch64) back-end of the Jasmin compiler.
//   rp[0..3] = ap[0..3] + bp[0..3]  (mod 2^256)
export
fn add256(reg u64 rp ap bp) -> reg u64 {
  reg u64 a b carry zero;
  reg bool cf;
  inline int i;

  a = [ap];
  b = [bp];
  cf, a = a + b;            // ADDS: sets the carry flag
  [rp] = a;
  for i = 1 to 4 {
    a = [ap + 8 * i];
    b = [bp + 8 * i];
    cf, a += b + cf;        // ADCS: add with carry
    [rp + 8 * i] = a;
  }
  zero = 0;
  carry = 0;
  _, carry += zero + cf;    // ADC with register operands only: carry = cf
  return carry;
}
```

Listing 1: `examples/jasmin/add256.jazz`

The compiler translates the program into an assembly file for the ARMv8-A target:

```
$ cd examples/jasmin
$ jasminc -arch armv8a -o add256.s add256.jazz
$ cat add256.s
```

```
        .global add256
        .p2align        2
        .type   add256, %function
add256:
        str     x30, [sp, #-16]!
        ldr     x3, [x1]
        ldr     x4, [x2]
        adds    x3, x3, x4
        str     x3, [x0]
        ldr     x3, [x1, #8]
        ldr     x4, [x2, #8]
        adcs    x3, x3, x4
        str     x3, [x0, #8]
        ldr     x3, [x1, #16]
```

```
        ldr     x4, [x2, #16]
        adcs    x3, x3, x4
        str     x3, [x0, #16]
        ldr     x3, [x1, #24]
        ldr     x4, [x2, #24]
        adcs    x3, x3, x4
        str     x3, [x0, #24]
        movz    x0, #0
        movz    x1, #0
        adc     x0, x1, x0
        ldr     x30, [sp], #16
        ret
        .ident  "Jasmin Compiler 2026.09.0"
        .section        ".note.GNU-stack", "", %progbits
```

Listing 2: AArch64 assembly generated by `jasminc` (`add256.s`)

The generated code follows AAPCS64, the procedure call standard of AArch64, which defines how a function receives its arguments and returns its result: the three pointer arguments arrive in the registers `x0`–`x2`, and the final carry is returned in `x0`. A C program can therefore call the function directly. The carry chain compiles to one `adds` instruction (addition that sets the carry flag) followed by `adcs` instructions (addition with carry). A C test program compares the result with a portable reference implementation, which computes with the 128-bit C type `unsigned __int128`, on 10^6 random inputs. Every seventh input is constructed so that the carry propagates through all four limbs:

```c
/* Native test harness: links the Jasmin-generated AArch64 assembly and
   compares add256 against a portable C reference on random inputs. */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern uint64_t add256(uint64_t *r, const uint64_t *a, const uint64_t *b);

static uint64_t ref_add256(uint64_t *r, const uint64_t *a, const uint64_t *b) {
  unsigned __int128 acc = 0;
  for (int i = 0; i < 4; i++) {
    acc += (unsigned __int128)a[i] + b[i];
    r[i] = (uint64_t)acc;
    acc >>= 64;
  }
  return (uint64_t)acc;
}

static uint64_t rnd64(void) {
  return ((uint64_t)rand() << 62) ^ ((uint64_t)rand() << 31) ^ (uint64_t)rand();
}

int main(void) {
  uint64_t a[4], b[4], r1[4], r2[4];
  srand(2026);
  for (int t = 0; t < 1000000; t++) {
    for (int i = 0; i < 4; i++) {
      a[i] = rnd64();
      b[i] = (t % 7 == 0) ? ~a[i] : rnd64();
    }
    if (t % 7 == 0) b[0] += 1;  /* a + b = 2^256: carry ripples through all limbs */
```

```
    uint64_t c1 = add256(r1, a, b);
    uint64_t c2 = ref_add256(r2, a, b);
    if (c1 != c2 || memcmp(r1, r2, sizeof r1) != 0) {
      printf("MISMATCH at test %d\n", t);
      return 1;
    }
  }
  printf("add256: 1000000 random tests passed\n");
  return 0;
}
```

Listing 3: `examples/jasmin/test_add256.c`

The C compiler links the C test program with the generated assembly file:

```
$ gcc -O2 -Wall -o test_add256 test_add256.c add256.s
$ ./test_add256
add256: 1000000 random tests passed
```

The same source file also compiles for the other targets of Jasmin, for example x86-64. In this case `jasminc` acts as a cross-compiler: it generates code for a processor other than the one on which it runs, and the Raspberry Pi cannot execute the result:

```
$ jasminc -arch x86-64 -o add256_x86.s add256.jazz
```

> **Caution.** The first version of the program ended with the statement `_, carry += 0 + cf;`, which the compiler rejected:
>
> ```
> "add256.jazz", line 21 (2-21):
> compilation error in function add256:
> asmgen: instruction ADC_64 is given incompatible args.
> Allowed args are:
>   [[reg]; [reg]; [reg]]
> ```
>
> The AArch64 instruction `ADC` accepts only registers as operands, not a constant (an *immediate*). Jasmin reports this restriction instead of inserting the necessary register moves implicitly, because each Jasmin instruction corresponds to exactly one machine instruction. The correction places the constant in a register (`zero = 0;`) and uses the statement `_, carry += zero + cf;`.

8.2 Test 2: constant-time verification with `jasmin-ct`

A conditional selection returns one of two values depending on a condition bit. If the condition bit is secret and the selection is implemented with a branch, the execution time and the memory accesses can depend on the secret. The checker `jasmin-ct` verifies that secret data influence neither the control flow nor the memory addresses of a program. For this purpose, the annotation `#[ct = "..."]` declares the *security type* of each argument and of the result: `public` (known to an attacker) or `secret`. The test file defines two functions that compute the same result: a branch-free selection, which combines the two values with a mask computed from the condition bit, and a leaky selection, which branches on the secret bit:

```
// Constant-time conditional selection: return a if c = 1, b if c = 0.
// The secret bit c must never influence control flow or memory addresses.

// Branch-free version: builds an all-ones / all-zeros mask from c.
#[ct = "secret * secret * secret -> secret"]
export
fn ct_select(reg u64 c a b) -> reg u64 {
```

```
  reg u64 mask r t;
  mask = 0;
  mask -= c;          // mask = 0xff..ff if c = 1, 0 if c = 0
  r = a & mask;
  t = b & !mask;
  r |= t;
  return r;
}

// Leaky version: branches on the secret bit (rejected by jasmin-ct).
#[ct = "secret * secret * secret -> secret"]
export
fn leaky_select(reg u64 c a b) -> reg u64 {
  reg u64 r;
  if (c == 1) { r = a; } else { r = b; }
  return r;
}
```

Listing 4: `examples/jasmin/ct_select.jazz`

The option `--slice` restricts the check to one function. The first command checks only `ct_select`; the second checks the whole file:

```
$ jasmin-ct --arch=armv8a --slice=ct_select ct_select.jazz; echo "exit=$?"
exit=0
$ jasmin-ct --arch=armv8a ct_select.jazz; echo "exit=$?"

"ct_select.jazz", line 22 (6-7):
constant type checker: c has type secret it needs to be public
exit=1
```

The checker accepts `ct_select` (exit code 0) and rejects `leaky_select` (exit code 1); its error message identifies the offending condition `c == 1` in line 22. The option `--speculative` additionally verifies speculative constant time (Spectre v1).

8.3 Test 3: EasyCrypt proofs

An EasyCrypt file states *lemmas*, each followed by a proof between the keywords `proof` and `qed`. A proof consists of *tactics*, commands that transform the statement to be proved (the *goal*) into simpler goals until none remains. EasyCrypt rejects the file if any goal remains open. This test checks three files.

SMT smoke test. The first file verifies that the standard library loads, that the tactic `smt()` can delegate goals to the SMT solvers through Why3, and that the tactic `ring`, which proves algebraic identities, is available:

```
(* Smoke test: arithmetic facts discharged by the SMT back-end (Why3). *)
require import AllCore StdRing.
import RField.

lemma add_comm_int (x y : int) : x + y = y + x.
proof. by smt(). qed.

lemma square_nonneg (x : int) : 0 <= x * x.
proof. by smt(). qed.

lemma distrib (a b c : int) : a * (b + c) = a * b + a * c.
proof. by ring. qed.
```

Listing 5: `examples/easycrypt/hello.ec`

Probabilistic relational proof. The one-time pad encrypts a message bit m as $m \oplus k$ with a uniformly random key bit k. It is perfectly secret: the ciphertext has the same distribution as a fresh uniform bit, whatever the message. The file states this property in pRHL as an equivalence (`equiv`) between two procedures, the real encryption `OTP.enc` and an ideal procedure `Ideal.enc` that returns a random bit. The postcondition `={res}` requires that the results of the two procedures be equal when their random choices are suitably paired; such a pairing, called a *coupling*, implies that the two results have the same distribution. The tactic `rnd` establishes the coupling: it pairs each key k of the real procedure with the value $m \oplus k$ of the ideal procedure. Because this pairing is a bijection between the two sets of random choices, it preserves probabilities, and the two results coincide:

```
(* One-time pad on booleans: the ciphertext is uniformly distributed,
   whatever the message.  Proved as a pRHL equivalence between the real
   encryption and an oracle that returns a fresh random bit. *)
require import AllCore Bool Distr DBool.

module OTP = {
  proc enc(m : bool) : bool = {
    var k;
    k <$ {0,1};
    return m ^^ k;
  }
}.

module Ideal = {
  proc enc(m : bool) : bool = {
    var r;
    r <$ {0,1};
    return r;
  }
}.

lemma otp_perfect_secrecy :
  equiv [OTP.enc ~ Ideal.enc : true ==> ={res}].
proof.
proc.
rnd (fun k => m{1} ^^ k) (fun r => m{1} ^^ r).
skip => /> &1.
smt(xorA xorK xor_false dbool1E supp_dbool).
qed.
```

Listing 6: `examples/easycrypt/otp.ec`

Negative test. A proof checker is useful only if it rejects false statements. The following file states a false lemma ($x + 1 = x$), which EasyCrypt must reject:

```
(* Negative test: a false statement must be rejected. *)
require import AllCore.

lemma wrong (x : int) : x + 1 = x.
proof. by smt(). qed.
```

Listing 7: `examples/easycrypt/wrong.ec`

The command `easycrypt compile` checks a file and reports success with exit code 0:

```
$ cd examples/easycrypt
$ easycrypt compile -no-eco hello.ec; echo "exit=$?"
exit=0
$ easycrypt compile -no-eco otp.ec; echo "exit=$?"
exit=0
$ easycrypt compile -no-eco wrong.ec; echo "exit=$?"
[critical] [wrong.ec: line 5 (7-16)] cannot prove goal (strict)
exit=1
```

The option `-no-eco` disables the cache of previously checked theories (`.eco` files) and thereby forces a complete check. On the Raspberry Pi 5, the checks took 1.2 s (`hello.ec`) and 2.6 s (`otp.ec`).

> **Note. Inspection of proof goals without an editor.** When a tactic fails, `easycrypt compile` reports only the error. To display the goal that remains at a given line, pass the file up to that line to the command-line interface of EasyCrypt:
>
> ```
> $ sed -n '1,12p' ct_select_proof.ec | easycrypt cli -emacs
> ...
> Current goal (remaining: 2)
> ----------------------------------------------------------------------
> a0 `&` - one `|` b0 `&` invw (- one) = a0
> ```
>
> For regular proof development, an interactive editor is more convenient: Proof General (Emacs) or the EasyCrypt extension for VS Code.

8.4 Test 4: translation from Jasmin to EasyCrypt and correctness proof

This test combines the two tools. The translator `jasmin2ec` converts the constant-time function `ct_select` of Test 2 into an EasyCrypt module (`M`) with one procedure per translated function. The option `--arch=armv8a` selects the eclib model of the ARMv8-A instructions, and the option `-f` selects the function to translate:

```
$ cd examples/jasmin
$ jasmin2ec --arch=armv8a -f ct_select -o ../easycrypt/CtSelect.ec ct_select.jazz
```

```
require import AllCore IntDiv CoreMap List Distr.

from Jasmin require import JModel_armv8a.

import SLH64.

module M = {
  proc ct_select (c:W64.t, a:W64.t, b:W64.t) : W64.t = {
    var r:W64.t;
    var mask:W64.t;
    var t:W64.t;
    mask <- (W64.of_int 0);
    mask <- (mask - c);
    r <- (a `&` mask);
    t <- (b `&` (invw mask));
    r <- (r `|` t);
    return r;
  }
}.
```

Listing 8: `examples/easycrypt/CtSelect.ec` (generated)

A proof in Hoare logic then establishes that the branch-free code computes a selection. The statement `hoare [ M.ct_select : P ==> Q ]` asserts that every execution of the procedure that starts in a state satisfying the precondition `P` ends in a state satisfying the postcondition `Q`. Here, the precondition requires $c \in \{0, 1\}$, and the postcondition requires that the result `res` equals a if $c = 1$ and b if $c = 0$. The proof treats the two cases separately and relies on the bit-level properties of the mask $0 - c$, which consists of either all ones or all zeros, using lemmas of `JWord.ec`:

```
(* Functional correctness of the Jasmin function ct_select, proved on the
   EasyCrypt model produced by jasmin2ec (file CtSelect.ec). *)
require import AllCore.
from Jasmin require import JModel_armv8a.
require import CtSelect.

lemma ct_select_correct (c0 a0 b0 : W64.t) :
  hoare [ M.ct_select :
          c = c0 /\ a = a0 /\ b = b0 /\ (c0 = W64.one \/ c0 = W64.zero)
          ==> res = if c0 = W64.one then a0 else b0 ].
proof.
proc; wp; skip => /> [-> | ->] /=.
- (* c = 1: mask = -1 = all ones, so a0 & mask = a0 and b0 & ~mask = 0 *)
  by rewrite W64.minus_one W64.andw1_s -W64.xor1w W64.xorwK W64.andw0 W64.orw0.
- (* c = 0: mask = 0, so a0 & mask = 0 and b0 & ~mask = b0 *)
  have -> : -W64.zero = W64.zero by ring.
  rewrite -W64.xor1w W64.xorw0 W64.andw1_s W64.andw0 W64.or0w.
  smt(W64.to_uint0 W64.to_uint1).
qed.
```

Listing 9: `examples/easycrypt/ct_select_proof.ec`

```
$ cd ../easycrypt
$ easycrypt compile -no-eco ct_select_proof.ec; echo "exit=$?"
exit=0
```

The proof is checked in 6.5 s on the Raspberry Pi 5; most of this time is spent loading the Jasmin eclib. Together with Test 2, this result establishes that `ct_select` is constant-time (`jasmin-ct`) *and* functionally correct (EasyCrypt). Because the Jasmin compiler is verified, both properties also hold for the generated AArch64 code.

8.5 Execution of all tests

The script `examples/run-tests.sh` rebuilds and checks all examples. It performs 11 checks, which divide the four tests above into their individual steps. For the two negative checks (the leaky selection and the false lemma), the script verifies that the tool fails as intended, with exit code 1:

```
#!/usr/bin/env bash
# Re-run every Jasmin / EasyCrypt test of the installation guide.
# Usage: ./run-tests.sh            (from the examples/ directory)
set -u
cd "$(dirname "$0")"
eval "$(opam env --switch=jasmin-ec)"

pass=0; fail=0
check() {  # check <expected-exit-code> <description> <command...>
  local want=$1 desc=$2; shift 2
  "$@" > /tmp/jasmin-ec-test.out 2>&1
  local got=$?
  if [ "$got" -eq "$want" ]; then
```

```
    echo "[PASS] $desc"; pass=$((pass + 1))
  else
    echo "[FAIL] $desc (exit $got, expected $want)"; cat /tmp/jasmin-ec-test.out
    fail=$((fail + 1))
  fi
}

echo "== Tool versions"
jasminc -version
echo "EasyCrypt $(easycrypt config 2>&1 | sed -n 's/^git-hash: //p')"
easycrypt config 2>&1 | grep '^known provers'

echo "== Jasmin"
cd jasmin
check 0 "jasminc: compile add256.jazz for ARMv8-A" \
  jasminc -arch armv8a -o add256.s add256.jazz
check 0 "gcc: link Jasmin assembly with C harness" \
  gcc -O2 -Wall -o test_add256 test_add256.c add256.s
check 0 "native run of add256 on the Raspberry Pi 5" ./test_add256
check 0 "jasminc: compile add256.jazz for x86-64 (cross target)" \
  jasminc -arch x86-64 -o add256_x86.s add256.jazz
check 0 "jasmin-ct: ct_select is constant-time" \
  jasmin-ct --arch=armv8a --slice=ct_select ct_select.jazz
check 1 "jasmin-ct: leaky_select is rejected" \
  jasmin-ct --arch=armv8a --slice=leaky_select ct_select.jazz
check 0 "jasmin2ec: extract ct_select to EasyCrypt" \
  jasmin2ec --arch=armv8a -f ct_select -o ../easycrypt/CtSelect.ec ct_select.jazz
cd ..

echo "== EasyCrypt"
cd easycrypt
check 0 "easycrypt: hello.ec (SMT smoke test)" easycrypt compile -no-eco hello.ec
check 0 "easycrypt: otp.ec (pRHL proof)" easycrypt compile -no-eco otp.ec
check 0 "easycrypt: ct_select_proof.ec (proof on extracted Jasmin)" \
  easycrypt compile -no-eco ct_select_proof.ec
check 1 "easycrypt: wrong.ec is rejected" easycrypt compile -no-eco wrong.ec
cd ..

rm -f /tmp/jasmin-ec-test.out
echo "== Summary: $pass passed, $fail failed"
[ "$fail" -eq 0 ]
```

Listing 10: examples/run-tests.sh

```
$ cd examples
$ time ./run-tests.sh | tee run-tests-output.txt
```

```
== Tool versions
Jasmin Compiler 2026.09.0
EasyCrypt r2026.07
known provers: Alt-Ergo@2.6.0, Alt-Ergo[BV]@2.6.0, Alt-Ergo[counterexamples]@2.6.0, CVC5@1.0.3, CVC5[
    ↪ counterexamples]@1.0.3, CVC5[strings]@1.0.3, CVC5[strings+counterexamples]@1.0.3, Z3@4.8.12, Z3[
    ↪ counterexamples]@4.8.12, Z3[noBV]@4.8.12
== Jasmin
[PASS] jasminc: compile add256.jazz for ARMv8-A
[PASS] gcc: link Jasmin assembly with C harness
[PASS] native run of add256 on the Raspberry Pi 5
[PASS] jasminc: compile add256.jazz for x86-64 (cross target)
[PASS] jasmin-ct: ct_select is constant-time
```

```
[PASS] jasmin-ct: leaky_select is rejected
[PASS] jasmin2ec: extract ct_select to EasyCrypt
== EasyCrypt
[PASS] easycrypt: hello.ec (SMT smoke test)
[PASS] easycrypt: otp.ec (pRHL proof)
[PASS] easycrypt: ct_select_proof.ec (proof on extracted Jasmin)
[PASS] easycrypt: wrong.ec is rejected
== Summary: 11 passed, 0 failed
```

Listing 11: Output of `run-tests.sh` on the Raspberry Pi 5

The complete test suite runs in about 15 s. Each line marked `[PASS]` corresponds to one check; a failed check is marked `[FAIL]` and makes the script exit with a nonzero code.

9 Summary

Table 7 summarizes the time and storage that the installation required.

Step	**Wall-clock time**	**Remarks**
apt packages	≈ 2 min	opam, build tools, Z3, CVC5
`opam init` and OCaml 4.14.2 switch	≈ 12 min	compiler built from source
Jasmin 2026.09.0	16 min	including Apron, PPL bindings, Menhir
EasyCrypt r2026.07 and Alt-Ergo 2.6.0	≈ 57 min	including Bitwuzla (C++), Why3, rebuild of Jasmin
Test suite (4 tests, 11 checks)	15 s	all checks pass
Storage used by `~/.opam`	4.0 GB	switch `jasmin-ec`: 3.9 GB

Table 7: Time and storage requirements on the Raspberry Pi 5 (8 GB RAM, microSD storage).

Recommendations.

- **Select matching versions.** Install the EasyCrypt release that Jasmin's `CHANGELOG.md` names ("Extraction as EasyCrypt code targets version . . . "), not the latest EasyCrypt release.
- **Reclaim storage before the build.** A margin of 6–8 GB is sufficient. On a full disk, builds fail with misleading error messages.
- **Verify prover binaries for arm64.** The upstream Z3 builds for arm64 are either mislabeled (4.12.4) or require glibc ≥ 2.38. On Bookworm, use Debian's Z3 together with Alt-Ergo from opam.
- **Observe the ARMv8-A operand rules.** Instructions such as `ADC` accept only register operands; when a program violates such a rule, `jasminc` reports the permitted operand forms.
- **Run long builds in a persistent session.** The `opam` process continues after a disconnection, but a terminal multiplexer such as `tmux` or `screen` keeps its output visible and allows reconnecting to it.

The second document of this repository (`../armv8-tests`) builds on this installation: it tests the ARMv8-A back end of Jasmin systematically on the Raspberry Pi 5.

A Complete command sequence

```
# 1. System packages
$ sudo apt-get update
$ sudo apt-get install -y opam build-essential m4 pkg-config unzip bubblewrap \
    libgmp-dev libpcre3-dev zlib1g-dev libmpfr-dev libppl-dev autoconf z3 cvc5

# 2. opam and OCaml
$ opam init -y --bare --disable-sandboxing
$ opam switch create jasmin-ec ocaml-base-compiler.4.14.2 -y
$ eval $(opam env --switch=jasmin-ec)

# 3. Jasmin
$ opam install -y jasmin.2026.09.0

# 4. EasyCrypt and Alt-Ergo
$ opam pin add -yn easycrypt https://github.com/EasyCrypt/easycrypt.git#r2026.07
$ opam install -y easycrypt alt-ergo.2.6.0
$ easycrypt why3config
$ easycrypt config

# 5. Tests
$ cd install-guide/examples && ./run-tests.sh
```

B Glossary

AAPCS64 Procedure Call Standard for the Arm 64-bit Architecture: the rules by which functions pass arguments, return results and preserve registers.

AArch64, ARMv8-A The 64-bit execution state and the version of the Arm architecture that the Cortex-A76 of the Raspberry Pi 5 implements; used interchangeably in this guide.

Back end The part of a compiler that generates code for one particular processor architecture (target).

Carry flag A processor flag that records whether the last addition produced a result too large for its register.

Constant time The property that secret data influence neither the control flow nor the memory addresses of a program, so that its execution time reveals nothing about the secrets.

Coq A proof assistant in which the correctness of the Jasmin compiler is proved.

Coupling In pRHL: a pairing of the random choices of two programs that preserves their probabilities.

Cross-compiler A compiler that generates code for a processor other than the one on which it runs.

eclib The EasyCrypt library of Jasmin, which models machine words and instructions for programs translated by `jasmin2ec`.

Formal verification The establishment of properties of a program by machine-checked mathematical proof, for all inputs.

Hoare logic A logic for statements of the form "if the precondition holds before the execution, the postcondition holds after it".

Immediate A constant operand that is encoded directly in an instruction.

Lemma, tactic, goal In EasyCrypt: a stated property, a proof step, and a statement that remains to be proved.

Limb One machine word of a multi-word integer.

opam, switch, pin The OCaml package manager; an independent installation of an OCaml compiler with its packages; the association of a package name with a specific source.

pRHL Probabilistic relational Hoare logic: a logic that relates the output distributions of two probabilistic programs.

Security type In `jasmin-ct`: the annotation of a value as `public` or `secret`.

SMT solver An automatic prover for logical formulas over arithmetic and other theories (here Alt-Ergo, CVC5 and Z3).

Why3 A platform through which EasyCrypt calls the SMT solvers.

C References

- Jasmin: `https://github.com/jasmin-lang/jasmin`
- Jasmin documentation: `https://jasmin-lang.readthedocs.io`
- EasyCrypt: `https://github.com/EasyCrypt/easycrypt`
- Formosa-Crypto Zulip chat (user community of Jasmin and EasyCrypt): `https://formosa-crypto.zulipchat.com`